

Module 4: Loops

Sarah Foss

Objectives

Write programs using **while**, **do-while**, and **for** loops, and understand their similarities and differences

Design effective loops, including controlling them with a sentinel value and minimizing numerical errors

Work with nested loops

Understand program control using **break** and **continue**

Iteration & Looping

A computer does simple operations extremely quickly.

If all programs consisted of simple statements and decisions as we have seen so far, then we would never be able to write enough code to use a computer effectively.

To make a computer do a set of statements multiple times we use **looping structures**.

A **loop** repeats a set of statements multiple times until some condition is satisfied.

Each time a loop is executed is called an **iteration**.

The **while** Loop

The most basic looping structure is the **while** loop.

A **while** loop continually executes a set of statements while a condition is **true**.

Syntax:

```
<initialization>
while (<condition>) {
    //statements
    <update>
}
```

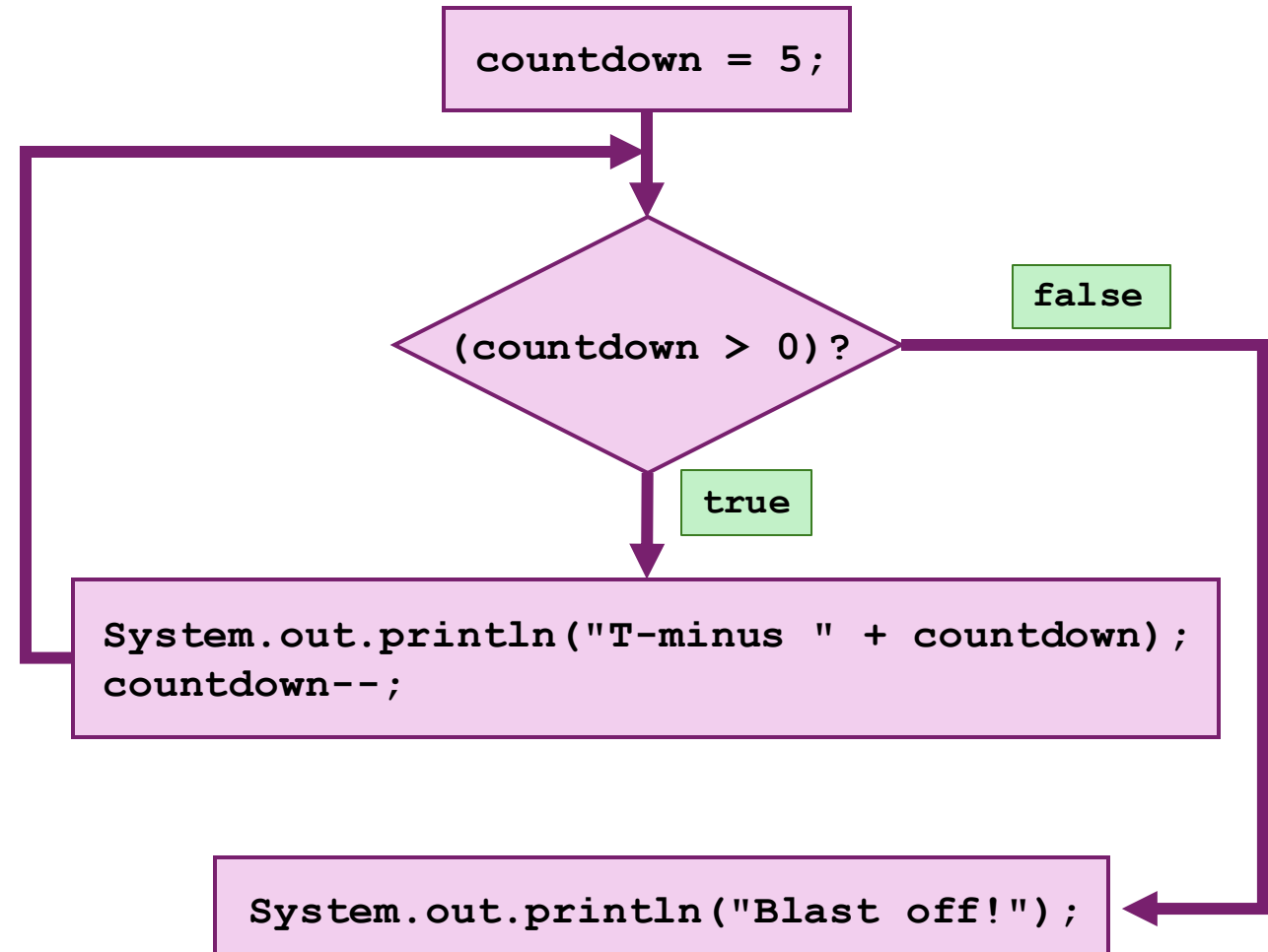
Example:

```
int j=0;
while (j <= 5){
    j++;
    System.out.println(j);
}
```

Question: What does this print?

The `while` Loop

```
int countdown = 5;
while (countdown > 0) {
    System.out.println("T-minus " + countdown);
    countdown--;
}
System.out.println("Blast off!");
```



Infinite Loops

An **infinite loop** is a loop that runs endlessly.

Causes of Infinite Loops:

- 1. Condition that is always true:** The loop's condition is written in a way that it will never evaluate to false.

```
while (true) {  
    System.out.println("This will print forever!");  
}
```

- 2. Improper update:** The variable controlling the loop isn't properly updated, so the loop never reaches its exit condition.

```
int i = 3;  
while (i > 0) {  
    System.out.println("i = " + i);  
    i++;  
}
```

Sentinel Value

The number of times a loop runs is often unknown in advance.

A special input value can be used to indicate when the loop should stop.

This value is called a **sentinel value**.

```
int number = 0;  
int sum = 0;
```

```
System.out.print("Enter numbers to sum. ");  
System.out.println("Enter -1 to stop.");
```

```
while (number != -1) {  
    System.out.print("Enter a number: ");  
    number = input.nextInt();  
  
    if (number != -1) {  
        sum += number;  
    }  
}
```

Aside: `Math.random()`

`Math.random()` is a method that generates a random double number between 0.0 (**inclusive**) and 1.0 (**exclusive**).

In other words, it returns a double value that is greater than or equal to 0.0 but less than 1.0.

```
(int) (Math.random() * 10)
```

Returns a random integer between 0 and 9.

```
1 + (int) (Math.random() * 20)
```

Returns a random integer between 1 and 20.

Guess That Number!

```
import java.util.Scanner;
public class GuessThatNumber {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Generate a random number between 0 and 100 (inclusive)
        int randomNumber = (int) (Math.random() * 101);
        int userGuess = -1;

        System.out.println("Guess the number between 0 and 100!");

        while (userGuess != randomNumber) {
            System.out.print("Enter your guess: ");
            userGuess = input.nextInt(); // Read the user's input

            if (userGuess < randomNumber) {
                System.out.println("Too low, try again!");
            } else if (userGuess > randomNumber) {
                System.out.println("Too high, try again!");
            } else {
                System.out.println("Congratulations! You've guessed the number.");
            }
        }
    }
}
```

Common Problems – **double** in Equality Check

Don't use floating-point numbers in an equality check in a loop because floating-point numbers are often imprecise.

This imprecision can lead to unexpected results when comparing values for exact equality.

For example:

```
double value = 0.0;
while (value != 1.0) {
    value += 0.1;
    System.out.println(value);
}
```

The do-while Loop

The do-while loop is a variation of the while loop.

Unlike the while loop, the do-while loop executes the loop body first before evaluating the continuation condition.

The do-while loop will always **run at least once**, regardless of the condition.

Syntax:

```
do {  
    //statements  
    <update>  
} while (<condition>);
```

The do-while Loop

```
Scanner input = new Scanner(System.in);  
  
int data;  
int sum = 0;  
  
do {  
    System.out.println("Enter an integer (enter 0 to stop)");  
    data = input.nextInt();  
    sum += data;  
} while(data != 0 );  
  
System.out.println("The sum is " + sum);
```

The **do-while** Loop Semicolon

In a **do-while** loop, the semicolon is an important part of the syntax.

It is placed after the **while** condition, signaling the end of the loop:

```
int i=0;
do {
    System.out.println("i is " + i);
    i++;
} while (i<10);
```



Correct

This is a different from a regular **while** loop, which does not use a semicolon after the condition.

```
int i=0;
while (i < 10) ;
{
    System.out.println("i is " + i);
    i++;
}
```



Logic Error

The **for** Loop

The most common type of loop is the **for** loop.

Syntax:

```
for (<initialization>; <condition>; <update>) {  
    //statements  
}
```

1. **initialization** section - is executed once at the start of the loop
2. **condition** section - is evaluated *before* every loop iteration to check for loop termination
3. **update** section - is evaluated *after* every loop iteration to update the loop counter

The `for` Loop

Although you can use almost any code in the three sections, there is a typical usage:

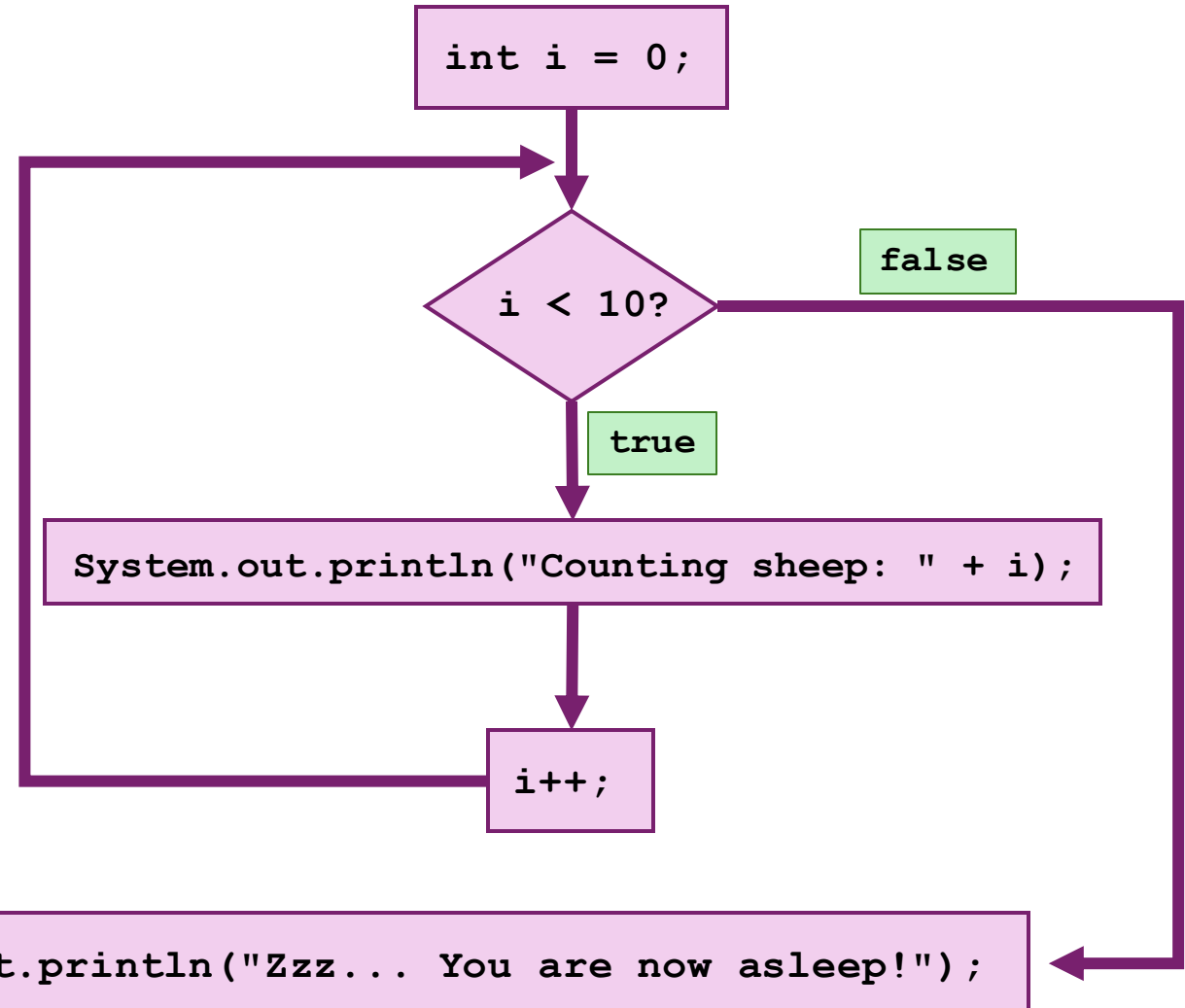
```
for (i = start; i < end; i++){  
    //statements  
}
```

Example:

```
int i;  
for (i = 0; i < 5; i++){  
    System.out.println(i);           // Prints 0 to 4  
}
```

The `for` Loop

```
for (int i = 0; i < 10; i++) {  
    System.out.println("Counting sheep: " + i);  
}  
System.out.println("Zzz... You are now asleep!");
```



for Loop and while Loop

The `for` loop is like a short-hand for the `while` loop:

```
int i=0;

while (i < 10) {
    System.out.println(i);
    i++;
}
```

```
int i;

for (i=0; i < 10; i++){
    System.out.println(i);
}
```

Infinite **for** Loops

Infinite loops can also occur with **for** loops:

Improper update:

```
for (int i = 3; i > 0; i++) {  
    System.out.println("i = " + i);  
}
```

No condition:

```
for ( ; ; ) {  
    System.out.println("This will print forever!");  
}
```

Common Problems – Off-by-one Error

A common error is to be "***off-by-one***". This occurs when you stop the loop one iteration too early or too late.

Example:

This loop was supposed to print 0 to 10, but it does not.

```
for (int i = 0; i < 10; i++)  
    System.out.println(i);    // Prints 0..9 not 0..10
```

Question: How can we fix this code to print 0 to 10?

Which Loop to Use?

Use a **for** loop when the number of iterations is known in advance, such as when you need to print a message 100 times.

Use a **while** loop when you don't know the number of repetitions, such as reading numbers until the input is 0.

Use a **do-while** loop when you want the loop to run at least once before checking the condition, regardless of the initial state of the condition.

Nested Loops

A loop can be nested inside another loop.

In nested loops, there is an outer loop and one or more inner loops. Each time the outer loop iterates, the inner loops are executed again from the beginning.

```
int count = 0;
for (int i = 0; i < 3; i++){
    for (int j = 0; j < 4; j++){
        count++;
    }
}
System.out.println(count);
```

Printing Stars

```
for (int i = 0; i < 3; i++){  
    for (int j = 0; j < 4; j++){  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

```
****  
****  
****
```

```
for (int i = 0; i < 4; i++){  
    for (int j = 0; j <= i; j++){  
        System.out.print("*");  
    }  
    System.out.println();  
}
```

```
*  
**  
***  
****
```

Performance Impact

Be mindful of performance when using nested loops, as the number of iterations grows quickly.

For example, consider the following code with three nested loops:

```
for (int i = 0; i < 10000; i++) {  
    for (int j = 0; j < 10000; j++) {  
        for (int k = 0; k < 10000; k++) {  
            System.out.print("*");  
        }  
    }  
}
```

In this case, the action inside the innermost loop is executed a total of one trillion times!

If the action takes 1 microsecond (one-millionth of a second), the total time to run this loop would exceed 277 hours!!!

break

You've used the **break** keyword in a **switch** statement before.

Similarly, **break** can also be used in a loop to instantly stop the loop's execution.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // Loop stops when i equals 5  
    }  
    System.out.println(i);  
}
```

Note: Instead of using **break** to terminate a loop prematurely, consider adjusting the loop's condition to control the flow.

continue

You can also use the **continue** keyword in a loop.

When it is encountered, it skips the rest of the current iteration and jumps to the next iteration of the loop.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        continue; // Skip the rest of the code when i equals 5  
    }  
    System.out.println(i);  
}
```

Careful!

While both **break** and **continue** have their uses, avoid relying on them to handle logic that could be better structured through more thoughtful loop conditions.

Excessive use can make your code harder to follow and debug.

Summary

- There are three types of loops: **while**, **do-while**, and **for**.
- Loop control has three components: **initialization**, **condition** check, and **update** of the control variable.
- The loop body contains the statements to be repeated.
- Each execution of the loop body is called an **iteration**.
- A **while** loop checks the condition first. If **true**, the body is executed; if **false**, the loop ends.
- An **infinite loop** runs endlessly without stopping.
- A **sentinel value** signals the end of a loop.
- A **do-while** loop executes the loop body first, then checks the condition.
- A **for** loop combines initialization, condition, and update into one line.

Summary

- **while** and **for** loops check the condition before running the loop body.
- **do-while** checks the condition after the loop body is run.
- **for** loops are used when the number of repetitions is known.
- **while** loops are used when the number of repetitions is unknown.
- **do-while** are used when you want the loop to run at minimum one time.
- Two keywords, **break** and **continue**, can be used in loops.
 - **break** exits the innermost loop.
 - **continue** skips the current iteration but continues the loop.